SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – PSEUDOCODE WRITING TASK

Course Code:	CSA12	**Course Title:**	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	**Assessment:**	Pseudocode Writing task
Weightage:	30%	**Total Marks:**	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	LAKSHMI DEVI .J	**Reg. No.:**	192524300
Section / Batch:	2025	**Date:**	26.08.26

Instructions to Students

- Identify the relevant concepts of DMA-based data transfer, vectored interrupts, I/O mapping techniques, bus arbitration, and hybrid I/O communication.
- Analyze the requirements of different I/O devices by considering CPU involvement, interrupt priority, latency, bandwidth, throughput, and transfer speed.
- Apply appropriate I/O techniques such as DMA, interrupt-driven I/O, vectored interrupts, memory-mapped I/O, I/O-mapped I/O, and bus arbitration to develop efficient solutions.
- Develop pseudocode algorithms for DMA data transfer, prioritized interrupt handling, dynamic I/O mapping selection, bus arbitration, and hybrid polling–DMA communication.
- Evaluate the proposed I/O mechanisms by interpreting CPU utilization, data transfer efficiency, interrupt response time, communication performance, resource allocation, and overall system suitability.

QUESTIONS

1. A real-time embedded system continuously receives large blocks of data from a high-speed I/O device. Write pseudocode to design a DMA-based data transfer algorithm that transfers data directly from the I/O device to main memory with minimal CPU involvement. The algorithm should initialize the DMA controller, configure source and destination addresses, specify the transfer size, initiate the transfer, and handle completion and error interrupts.

Purpose: Transfer a large block of data directly from a high-speed I/O device to main memory with minimum CPU involvement.

```
ALGORITHM DMA_Data_Transfer

START

    Initialize DMA Controller

    Set DMA Source Address
        SOURCE ← I/O_DEVICE_ADDRESS

    Set DMA Destination Address
        DESTINATION ← MAIN_MEMORY_ADDRESS

    Set Transfer Size
        SIZE ← DATA_BLOCK_SIZE

    Set Transfer Direction
        DIRECTION ← IO_TO_MEMORY

    Configure DMA Mode
        MODE ← BLOCK_TRANSFER
```

```
    Enable DMA Completion Interrupt
    Enable DMA Error Interrupt

    Start DMA Transfer

    WHILE DMA Transfer is ACTIVE
        CPU performs other tasks
    END WHILE

    WHEN DMA Interrupt occurs

        IF Interrupt_Type = COMPLETION THEN
            Clear DMA Interrupt
            Verify Transfer Status
            Mark Data Block as READY
            Notify Application
        END IF

        IF Interrupt_Type = ERROR THEN
            Read DMA Error Status
            Stop DMA Transfer
            Clear Error
            Report Error
            Restart or Recover DMA Transfer
        END IF

END
```

Flow

```
Initialize DMA
      ↓
Set Source Address
      ↓
Set Destination Address
      ↓
Set Transfer Size
      ↓
Configure DMA
      ↓
Start Transfer
      ↓
CPU performs other work
      ↓
DMA completes
      ↓
Interrupt
      ↓
Check Status
   ↙        ↘
Success     Error
  ↓           ↓
Process      Recover
Data         Transfer
```

Key point: The CPU only configures the DMA controller and handles completion/error interrupts. The actual bulk data transfer is performed by DMA.

2. An embedded system is connected to multiple I/O devices, each capable of generating interrupts with different priority levels. Develop pseudocode for an interrupt handling mechanism that identifies and prioritizes vectored interrupts, determines the interrupting device, and dispatches the appropriate interrupt service routine. The algorithm should ensure that high-priority interrupts are serviced efficiently while maintaining proper handling of lower-priority requests.

   Purpose: Identify the interrupting device, prioritize interrupts, and execute the appropriate ISR.

```
ALGORITHM Vectored_Interrupt_Handler

START

   Initialize Interrupt Controller

   Create Interrupt Vector Table

   Assign Priority to Each Device

   Enable Interrupts

   WHILE System is RUNNING

      IF Interrupt Request EXISTS THEN

         Read Pending Interrupts

         Find Highest Priority Interrupt

         DISABLE lower-priority interrupts temporarily

         VECTOR ← Get Interrupt Vector

         DEVICE ← Identify Device using VECTOR

         Save CPU Context

         IF DEVICE = HIGH_PRIORITY THEN
            Execute High_Priority_ISR
         ELSE IF DEVICE = MEDIUM_PRIORITY THEN
            Execute Medium_Priority_ISR
         ELSE
            Execute Low_Priority_ISR
         END IF

         Clear Interrupt Request

         Restore CPU Context

         Re-enable appropriate interrupts

      ELSE

         Continue Normal Execution

      END IF

   END WHILE

END
```

Interrupt Vector Table

Vector Number	Device	Priority
Vector 0	Timer	High
Vector 1	Data Acquisition	High
Vector 2	Network	Medium
Vector 3	Keyboard	Low
Vector 4	Sensor	Low

Nested Interrupt Handling
Low Priority ISR
↓
High Priority Interrupt
↓
Save Low Priority Context
↓
Execute High Priority ISR
↓
Restore Low Priority Context
↓
Continue Low Priority ISR
Key point: Vectored interrupts allow the CPU to directly locate the correct ISR, while priority handling ensures urgent devices are serviced first.

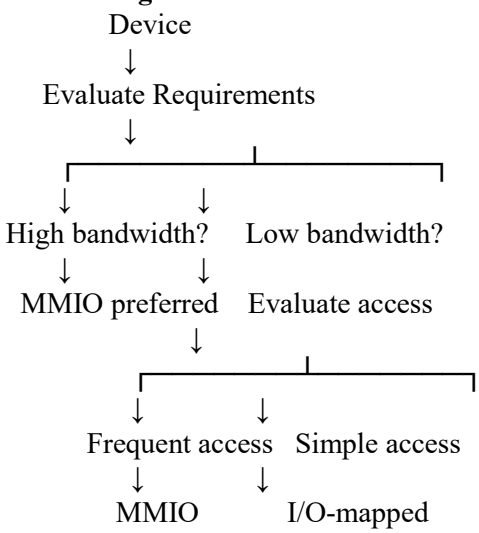
3. A computer system communicates with different I/O devices having varying latency, bandwidth, and datatransfer requirements. Write pseudocode for a dynamic I/O selection mechanism that chooses between memory-mapped I/O and I/O-mapped I/O based on the performance requirements of each device. The algorithm should evaluate latency and bandwidth requirements and select the mapping technique that provides efficient communication.

Purpose: Dynamically select **memory-mapped I/O (MMIO)** or **I/O-mapped I/O (PMIO)** according to device requirements.

```
ALGORITHM Dynamic_IO_Selection

START

  Read Device Information

  LATENCY ← Device Latency Requirement
  BANDWIDTH ← Device Bandwidth Requirement
  ACCESS ← Device Access Requirement

  IF LATENCY = HIGH_PRIORITY THEN

    Select MEMORY_MAPPED_IO

  ELSE IF BANDWIDTH > HIGH_BANDWIDTH_THRESHOLD THEN

    Select MEMORY_MAPPED_IO

  ELSE IF ACCESS = FREQUENT THEN

    Select MEMORY_MAPPED_IO

  ELSE

    Select I/O_MAPPED_IO

  END IF

  Configure Selected I/O Mapping

  Map Device Registers

  Initialize Device

  WHILE Device is ACTIVE
```

```
        Perform I/O Operation

        Monitor Latency and Bandwidth

        IF Performance is NOT Acceptable THEN
           Re-evaluate I/O Mapping
           Switch to More Suitable Technique
        END IF

    END WHILE

END
```

Selection Logic



Comparison

Requirement	Preferred Technique
Low latency	Memory-mapped I/O
High bandwidth	Memory-mapped I/O
Frequent register access	Memory-mapped I/O
Simple/low-frequency device	I/O-mapped I/O
Separate I/O address space	I/O-mapped I/O

Key point: The algorithm evaluates the device requirements before selecting the most appropriate mapping technique.

4. A computer system contains multiple devices that simultaneously request access to high-speed communication interfaces such as PCIe, USB, and Thunderbolt. Design pseudocode for a bus arbitration algorithm that manages competing access requests and allocates the appropriate interface to each device. The algorithm should consider device priority, bandwidth requirements, interface availability, and fairness to ensure efficient utilization of communication resources.

Purpose: Manage multiple devices requesting PCIe, USB, or Thunderbolt resources while maintaining priority, bandwidth, interface availability, and fairness.

```
ALGORITHM Bus_Arbitration

START

    Initialize PCIe, USB, and Thunderbolt Interfaces

    Initialize Device Request Queue

    WHILE System is RUNNING

        Collect all Device Requests
```

```
        FOR each Request

            Read Device Priority
            Read Bandwidth Requirement
            Check Interface Availability

        END FOR

        Remove Requests whose Interface is unavailable

        IF High Priority Request EXISTS THEN

            Select highest-priority request

        ELSE

            Select request using Fair Scheduling

        END IF

        Check Required Bandwidth

        IF Available_Bandwidth >= Requested_Bandwidth THEN

            Allocate Interface

            Grant Bus Access

            Start Data Transfer

        ELSE

            Place Request in Waiting Queue

        END IF

        Monitor Transfer

        IF Transfer Completed THEN

            Release Interface

            Update Device Usage

            Move to next waiting device

        END IF

        Apply Round-Robin/Fairness Policy

        Prevent any device from waiting indefinitely

    END WHILE

END
```

Arbitration Process

```
Device Requests
    ↓
```

Check Interface Availability
↓
Check Priority
↓
Check Bandwidth
↓
Select Device
↓
Allocate Interface
↓
Transfer Data
↓
Release Interface
↓
Serve Next Device

Example

Device A → PCIe → High Priority
Device B → USB → Medium Priority
Device C → Thunderbolt→ High Priority
Device D → USB → Low Priority

The arbiter considers:

1. **Priority**
2. **Required bandwidth**
3. **Interface availability**
4. **Current interface usage**
5. **Waiting time**

Round-robin scheduling can be used among devices having the same priority to prevent starvation.

Key point: The arbitration mechanism balances performance and fairness rather than allowing one device to continuously occupy the communication interface.

5. A real-time embedded system communicates with both low-speed devices, such as simple sensors, and high-speed devices, such as data acquisition units and storage devices. Write pseudocode for a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupt-driven communication for high-speed devices. The algorithm should identify the device type, select the appropriate I/O technique, initiate data transfer, and handle completion and error conditions while minimizing CPU involvement.

Purpose: Use polling for low-speed devices and DMA + interrupts for high-speed devices.

```
ALGORITHM Hybrid_IO_Communication

START

   Initialize All I/O Devices

   FOR each Device

      Identify Device Type

      IF Device Type = LOW_SPEED THEN

         Select POLLING

         Set Polling Interval

      ELSE IF Device Type = HIGH_SPEED THEN

         Select DMA
```

```
        Enable Completion Interrupt
        Enable Error Interrupt

    END IF

END FOR


WHILE System is RUNNING

    FOR each Device

        IF Device Type = LOW_SPEED THEN

            Check Device Status

            IF Data Available THEN
                Read Data
                Process Data
            END IF


        ELSE IF Device Type = HIGH_SPEED THEN

            IF DMA is NOT ACTIVE THEN

                Configure DMA
                Set Source Address
                Set Destination Address
                Set Transfer Size

                Start DMA Transfer

            END IF

        END IF

    END FOR


    IF Interrupt Received THEN

        Identify Interrupt Source

        IF Interrupt Type = DMA_COMPLETION THEN

            Verify Transfer
            Mark Data as READY
            Clear Interrupt

        ELSE IF Interrupt Type = DMA_ERROR THEN

            Read Error Status
            Stop DMA
            Recover or Restart Transfer
            Clear Error
```

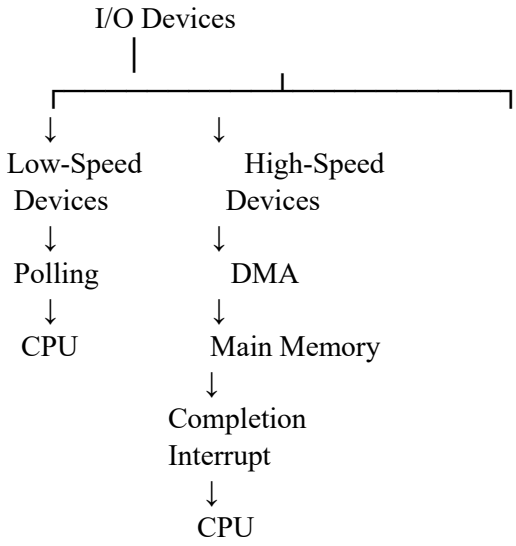
END IF

END IF

END WHILE

END

Hybrid Architecture



Comparison

Device Type	Technique	CPU Involvement
Low-speed sensor	Polling	Moderate
Keyboard/simple sensor	Polling	Low
Data acquisition unit	DMA + interrupt	Very low
High-speed storage	DMA + interrupt	Very low
High-speed network device	DMA + interrupt	Very low

Conclusion

The hybrid mechanism uses **polling only where the data rate is low**, avoiding unnecessary interrupt overhead. High-speed devices use **DMA with interrupt-driven completion**, allowing large amounts of data to move directly to memory while keeping CPU involvement minimal.

Code of Conduct

I J. LAKSHMI DEVI (192524300) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)

Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Q. No	Problem Understanding (20)	Algorithm Design (30)	Use of Control Structures (20)	Pseudocode Structure (10)	Completeness (20)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	**/30**	**/ 20**	**/ 10**	**/ 20**	**/ 100**

Course Coordinator **Faculty Incharge**